

A Machine Learning Approach to Twitter Sentiment Analysis

Model & Utility Application Implementation

CSCE 3602

Kirolous Fouty
Computer Engineering
Department
American University in Cairo
Cairo, Egypt
kirolous_fouty@aucegypt.edu

Mohamed Sabry
Computer Engineering
Department
American University in Cairo
Cairo, Egypt
momo12320@aucegypt.edu

1 Introduction

In today's digital age, the ability to understand and interpret human emotions through text has become increasingly valuable. Whether it's gauging customer satisfaction from reviews, monitoring social media sentiment, or enhancing user experience through personalized interactions, sentiment analysis plays a pivotal role. The Sentiment Analysis Tool project aims to harness the power of machine learning and natural language processing to provide an intuitive and efficient way to analyze the sentiment of textual data.

The Sentiment Analysis Tool is designed to be both powerful and accessible, making it an ideal solution for businesses, researchers, and individuals looking to gain insights from textual data. This report delves into the technical aspects of the project, detailing the architecture, implementation, and functionalities that make this tool a standout in the realm of sentiment analysis applications.

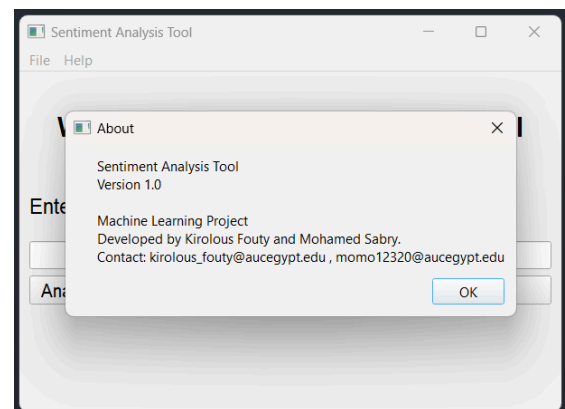
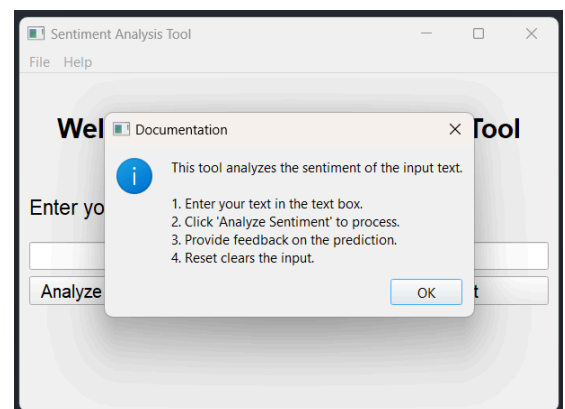
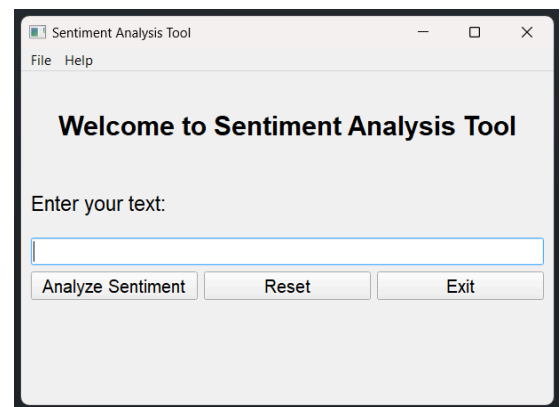
2 Application

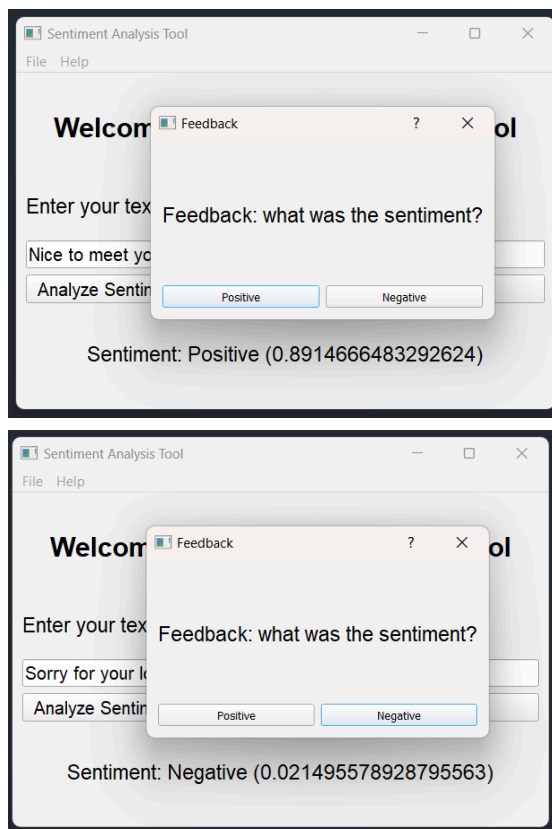
- **Overview:** The Sentiment Analysis Tool is a desktop application designed to analyze the sentiment of user-provided text. The application leverages machine learning models to predict whether the sentiment of the text is positive or negative. Additionally, the application allows users to provide feedback on the predictions, which can be used to further train and improve the model.
- **Technologies used:**
 - **Python:** The primary programming language used for the application.
 - **PyQt5:** A set of Python bindings for Qt libraries, used to create the graphical user interface (GUI).

- **FastAPI:** A modern, fast (high-performance) web framework for building APIs with Python.
- **Pydantic:** Used for data validation and settings management.
- **scikit-learn:** A machine learning library used for text vectorization and model training.
- **spaCy:** An open-source software library for advanced natural language processing.
- **NLTK:** The Natural Language Toolkit, used for text processing tasks.
- **Uvicorn:** A lightning-fast ASGI server used to serve the FastAPI application.
- **Requests:** A simple HTTP library for Python, used to send HTTP requests to the FastAPI endpoints.
- **Structure:**
 - **Backend (FastAPI)**
 - **Endpoints:**
 - /predict/: Accepts text input and returns the predicted sentiment.
 - /learn/: Accepts text input along with feedback (label) and updates the model with the new data.
 - **Models:**
 - InputData_Predict: Defines the schema for prediction requests.
 - InputData_Train: Defines the schema for training requests.
 - **Preprocessing Functions:** Functions to clean and normalize text data.
 - **Neural Network Classifier:** A custom neural network

implementation for sentiment analysis.

- **Frontend (PyQt5)**
 - **Main Window:** The primary interface where users can input text, analyze sentiment, and view results.
 - **Feedback Dialog:** A popup dialog that allows users to provide feedback on the prediction.
- **Workflow:**
 - **User Input:** The user enters text into the input field and clicks the "Analyze Sentiment" button.
 - **Prediction Request:** The application sends a POST request to the /predict/ endpoint with the input text.
 - **Prediction Response:** The backend processes the text, predicts the sentiment, and returns the result.
 - **Display Result:** The application displays the sentiment result in the result label.
 - **Feedback:** The user can provide feedback on the prediction through a popup dialog.
 - **Learning Request:** The application sends a POST request to the /learn/ endpoint with the input text and feedback.
 - **Model Update:** The backend updates the model with the new data and feedback.
- **Error Handling:**
 - **Input Validation:** The application checks if the input text is empty or exceeds the character limit before sending a request.
 - **Request Errors:** The application handles HTTP request errors and displays appropriate error messages to the user.
- **Conclusion:**
 - The Sentiment Analysis Tool is a comprehensive application that combines a user-friendly interface with powerful backend processing. It allows users to analyze the sentiment of text and provide feedback to improve the model over time. The use of modern technologies like FastAPI, PyQt5, and machine learning libraries ensures high performance and scalability.
- **Screenshots:**





3 System Architecture

Input Layer: Accepts pre-processed tweets (either as sequences of word embeddings or an embedding layer directly integrated into the MLP). Consider a maximum tweet length, such as 280 tokens (aligned with Twitter's character limit).

Neural Network Design:

- **Layers:** Our neural network design is based on the findings of the previous milestones. We found that a neural network with Relu activation functions performed well, so our final design is a 3 layer design. The first layer takes the number of text features in the dataset (242935) and produces 256 outputs and has a Relu activation function. The second is the same, but takes 256 and outputs 128, while the third takes 128 and outputs 64. The final layer is different from the rest, it takes 64 inputs and produces one output using a sigmoid activation function.
- **Optimization:** We implemented an Adam optimizer from scratch to go with the model. The optimizer helps in smoothing out the learning

process by removing noise and making the updates more productive.

- **Batching:** The model uses batching and sparse matrices since the data is huge and uses up a lot of memory in training. On a 32Gb system at 8192 batch size, it takes up nearly all of the system memory. For the application, we will use a batching size of 5 to fit the learning threshold.
- **Model performance:** The model requires a long time to train over all of the data. However, we managed to train it over 80% of the data and when testing with the remaining 20%, we got an accuracy of 0.7801823181527103 approximately 78%.

3 APIs

- **Predict Endpoint:** There are 2 API endpoints in the backend, one for predicting and one for learning. For the learning backend, it receives a JSON file with the tweet text and returns the probability of the tweet belonging to a certain class. The end-point processes the text by applying the same preprocessing code we did in the previous milestones and using the predict2 function of the model which returns a probability rather than a prediction.
- **Learn Endpoint:** This endpoint takes the input text as well as the label before processing them and storing them in a list. Once a threshold is passed (10), the model is fitted on the new samples.

4 Preprocessing Components

- **Input Preprocessing:** As shown in the previous milestones, text is processed in the following way. First of all, we load up the Countvectorizer values that we used to process the data of the dataset to a countvectorizer instance. We then apply the normalized corpus function that we used on the original dataset to process the text on the new text input. It expands contractions, removes nontext data, and removes user mentions. We then apply the fitted count vectorizer to it. We then feed this input to the model so that it returns a prediction.